

3D積付ソルバ(NEDO Challenge コンテスト2)天井突破の徹底調査:accessibility制約を第一級に据えた再定式化の提案

TL;DR

敗因の診断は正しい。あなたのソルバの天井(public 58.5)は「探索の詰め」の問題ではなく「定式化」の問題であり、搬入経路(transport-path)=到達可能性(accessibility)を後付けペナルティではなく構築規則の第一級市民として扱う再定式化が唯一の本質的な打ち手。文献上、reachability制約は「厳しくするほど充填率が落ちる」ことが実証済み(Junqueira et al. 2012、Bonet Filella et al. 2023が引用)であり、あなたの FRONT_WALL/TRANSPORT_MARGINが -11/-15 で失敗したのは理論通り。鍵は「厳しくする」ことではなく「wall-building(層/壁構築)+ front-to-back順序保証」という構造で accessibility を"無料で"満たす構築規則に切り替えること。

残り約5週間・2コア・GPUなしなら、最有力の1手は「wall-building/maximal-space 再構築 + reachability を厳格に満たす front-to-back 構築規則」への差し替え(Parreño et al. 2008 の maximal-space GRASP、Jiao et al. 2023 の CLP-RLS wall-building tree search が直接の下敷き)。DRL(PCT/Attend2Pack 等)は理論的には本命だが、GPUなし・5週間では学習が回らず非現実的。ILP/CP は 180s では中規模までしか解けず単独では非推奨だが、部分問題(層内2D配置)には有効。

凍結ではなく限定的継続を推奨。ただし投資は上記1本に集中し、2週間で強い信号(悪化シーンほぼゼロ)が出なければ現行 58.5 提出物を防御確定して撤退、というストップロスを設定するのが期待値最大。

Key Findings

1. 問題の本質:これは "container loading with unloading/reachability constraints" である

あなたの搬入L字経路デッドロックは、OR文献で確立された **unloading constraints(別名 sequential / rear / LIFO constraints)** そのものである。Bonet Filella, Trivella & Corman (2023, European Journal of Operational Research 308(1):336-352、所属 ETH Zürich / University of Twente) は、この制約に4種類があると整理する:(i) above constraints、(ii) visibility constraints、(iii) **reachability constraints**、(iv) separation constraints。あなたのケースは「荷物を前面からL字軌道で搬入する際に既配置物と干渉してはならない」= reachability constraints に該当する。

決定的に重要な既知の結果:**reachability(到達距離パラメータ δ)**を厳しくするほど充填効率が下がることが Junqueira et al. (2012) で示されており、Bonet Filella et al. (2023) はこれを次のように引用している (verbatim): "*Junqueira et al. (2012a) study reachability constraints when varying the parameter representing the distance that can be reached, showing that as these constraints become stricter, the packing often becomes less efficient.*" これはあなたの FRONT_WALL(-11)、TRANSPORT_MARGIN(-15)、DRIFT_MODEL(-11)がすべて「制約を厳しくして充填を削った」ために失敗した現象を、文献が事前に予言していたことを意味する。つまり「もっと厳しいフィルタ」路線は数学的に袋小路。正しい方向は「厳しくせずに構造で満たす」。

さらに Bonet Filella et al. (2023) は、彼ら自身の解法を **"a heuristic framework based on a randomized extreme-point constructive phase and a subsequent improvement phase. The latter phase iteratively destroys regions in the packing space where high penalties originate, and reconstructs them"** (verbatim)と記述している。これはあなたの現行構成(Extreme Point 貪欲構築 + 末尾K destroy の PACK_LNS)が**文献主流の骨格と同型**であることの裏付け=つまり、あなたは「標準的な良い解法」を作り切った上で天井に当たっている。だからこそ次の一手は骨格そのものの変更(下記2)である。

2. accessibility を**"無料で"**満たす構築規則:wall-building と front-to-back

Jiao, Huang, Song, Li & Wang (2023), **"Container loading problem based on robotic loader system: An optimization approach"**, Expert Systems with Applications, [ScienceDirect](#)

Vol. 236, 121222, DOI 10.1016/j.eswa.2023.121222(オンライン公開 2024-02-01) は、あなたの問題とほぼ同型。ロボットが伸縮ローラーラインと共にトラック内に進入して積むため、彼らは2つの新制約を導入(要旨 [ScienceDirect](#)

verbatim: *"two new constraints related to RLS: pallet continuity constraint and robot position constraint"*):

pallet continuity constraint(同一パレットの荷物の積付順序は連続)

robot position constraint(ロボットは前から後ろへ増分的にのみ積める) ← あなたの搬入経路制約と完全同型
解法は要旨 verbatimで **"we develop a tree search approach based on wall-building"**。目的関数が独特で「全荷物を積むのに必要な長さ(loading length)の最小化」であり、体積最大化ではなく [ScienceDirect](#)

accessibility を軸に据えている。この「front-to-back を構築規則としてハード保証し、その中で層を最適化する」設計が、あなたの「奥から前へ層で積む」路線の正しい発展形。

Parreño, Alvarez-Valdés, Oliveira & Tamarit (2008), **"A Maximal-Space Algorithm for the Container Loading Problem"**, INFORMS Journal on Computing 20(3):412–422, DOI 10.1287/ijoc.1070.0254 と続く **hybrid GRASP/VND (Annals of Operations Research, 2010)** は、maximal-space(空きを非交和の極大直方体集合で表現)+ブロック/層構築の決定版。要旨 [ResearchGate](#)

verbatim: *"tested over the complete set of Bischoff and Ratcliff problems, ranging from weakly heterogeneous to strongly heterogeneous cargo, and outperforms all the known nonparallel approaches that, partially or completely, have used this set of test problems."* C++実装で高速。あなたの Extreme Point ベースの候補生成を maximal-space + 層(wall/layer)構築に置き換える下敷きになる。

Männel & Bortfeldt (2016, 2018) は multi-drop で「単純なLIFOだけでは再ハンドリングを排除できず、追加制約が必要」と示す。あなたの「素朴な後方密着(FRONT_WALL)だけでは駄目」という実測と整合。

[ScienceDirect](#)

Crainic, Perboli & Tadei (2008), **"Extreme Point-Based Heuristics for Three-Dimensional Bin Packing"**, INFORMS JoC 20(3):368–384 = あなたが既に使っている Extreme Point の原典。accessibility は扱わないので、これ単体では天井の理由になっている。

3. accessibility を precedence graph で明示的に扱う学習/探索系

Hu, Xu, et al. (2020), **"TAP-Net: Transport-and-Pack using Reinforcement Learning"**, ACM Transactions on Graphics 39(6)(SIGGRAPH Asia)、arXiv 2009.01469、GitHub: Juzhan/TAP-Net。これは決定的に関連。彼らは「obstruction and accessibility constraints」を **precedence graph** で表現し、3種のエッジ(TB=Top Block、LAB/RAB=Left/Right Access Block)で「Rを置くまでOは搬入/回転できない」を明示。あなたの TOPO_REORDER(DAG topo-sort)はこの発想の簡易版で、net -4 に留まったのは「DAGだけ作って構築

規則と統合しなかった」ため。TAP-Netの教訓は、precedenceを順序制約としてだけでなく配置ヒートマップ(height map)と同時にネットワークに入れて共最適化する点にある。

Zhao, Yu & Xu (2022, ICLR), "Learning Efficient Online 3D Bin Packing on Packing Configuration Trees" (PCT), GitHub: alexfrom0815/Online-3D-BPP-PCT。 オンライン3D-BPPのDRL決定版。候補配置を木のleaf nodeとして表現し、GAT(graph attention)で状態を符号化。連続空間・実務制約に強い。ただし学習にGPUが要る。 [arXiv](#)

Zhao, Zhu, Xu, Huang & Xu (2025), "Deliberate Planning of 3D Bin Packing on Packing Configuration Trees" (arXiv 2504.04421)。 PCTに**テスト時のdeliberate planning(木探索)**を足す。重要な知見:著者らは「純粋に学習側(訓練時)の安定性制約だけに頼ると、テスト時に不安定配置が時々混じるのは不可避」と述べ、**テスト時の物理シミュ検証は常に必要**と結論。あなたの「offline幾何が実機を30-50%過大評価」問題に対する文献側の答えは「クリティカルな配置だけ実物理で検証」路線を支持する。

Zhang, Zi & Ge (2021), "Attend2Pack" (arXiv 2107.04333) は sequence policy と placement policy を分離して同時学習。あなたの「順序」と「配置」の分離設計に理論的裏付けを与えるが、やはり学習前提。

4. 貪欲の局所最適脱出:あなたの失敗は文献の"典型的失敗モード"

短い貪欲ロールアウトを体積でランクすると **accessibility** が悪化(あなたのLook-ahead Beam depth1 -23%/depth2 -32%)は、評価関数が間違っている典型例。**Ha et al. (2017), "An online packing heuristic for the three-dimensional container loading problem in dynamic environments and the physical internet"** や近年のMCTS系が示す正しい設計は、ロールアウトを**「体積」ではなく「最後まで転がしたときの num_placed / デッドロックまでの手数」で評価する**こと。

"Effective Online 3D Bin Packing with Lookahead Parcels Using Monte Carlo Tree Search" (KDD 2026, arXiv 2601.02649) は、lookahead付きオンライン3D-BPPを **MPC(model predictive control)**問題として定式化しMCTSで解く。核心的な設計:(a) 終端でRL criticの価値推定 $V(s)$ を使う、(b) 「個々の配置がもたらす**長期的な空間浪費**」を罰する **auxiliary reward** を導入。これはあなたの corridor_penalty(近視眼的)を「長期空間浪費ペナルティ」に置き換える具体的レシビ。ただし critic の学習にGPUが要る点は注意。

Limited Discrepancy Search (Harvey & Ginsberg 1995) はあなたのLDBCの原典。あなたのLDBC失敗(偽デッドロック乱発)は「discrepancy を位置除外で表現した」ため。文献的に正しいLDSは**順序(sequence)上の discrepancy** に限定し、位置は貪欲プランナに任せる(あなたのCDRに近いが、CDRの失敗は「短予算offlineシミュでstallが起きない」点)。→後述の推奨で、offlineシミュを分位点評価に変える設計で解決可能。

ALNS(Ropke & Pisinger 2006) のVRP向けオペレータ大規模メタ分析(Voigt, "A Review and Ranking of Operators in ALNS for VRPs", EJOR 2024/SSRN 4551887)によれば、**sequence-based removal(連続した並びを外す)removal** オペレータが最も効果的、insertionは **regret(先読み)**系が最強。あなたのPACK_LNS(末尾K)は最弱クラスのランダム除去。sequence-based removal + regret insertion に変えれば、同じLNS骨格でも改善余地がある。

5. sim-to-real ギャップ(offline幾何が実機を過大評価)

Zhao et al. (2022), "Learning practically feasible policies for online 3D bin packing", Science China Information Sciences は、擬似静的平衡(quasi-static equilibrium)による高速安定性推定を提案。物理シミュを毎回回さず安定性を近似する手法で、8s/step 予算に収まる可能性がある。

RoboBPP (arXiv 2512.04415) はPyBulletで200ステップ回して線形/角速度の最大値から安定性報酬を算出。あなたの place_item(約300ステップ)と同型。この論文の教訓は「テスト時シミュは必須だが全配置に回すと遅い」→**クリティカル配置(大型・item_idx=0)だけ実物理チェック**という選択的検証が現実的。

文献の一致した結論:**offlineを一律悲観化するのは逆効果(あなたのDRIFT_MODEL -11 と整合)。代わりに「期待値ではなく分位点(例:悲観的な下側タイルの沈降量)で危険な配置だけを評価」または「学習した残差(sim-to-real gap)モデルをクリティカル配置だけに適用」**が推奨経路。

6. 上位60+ 勢が使っている可能性が高い手法(制約下で実装可能なものに限定)

NEDO Challenge コンテスト2(SIGNATE、2026-07-07開始、2026-10-19締切)の公開解法記事は現時点で確認できなかった(コンペ進行中のため上位解法は未公開)。ただし類似コンペ・文献から、2コア・GPUなし・offline 180s の制約で 60+ を狙える現実的アプローチは:

maximal-space/wall-building 構築 + reactive GRASP(RCL適応) — Parreño系。純CPU・高速・実装枯れ。最有力。

front-to-back を厳格保証する層構築 + 層内2D配置をCP-SAT/ILPで最適化 — Jiao 2023 の思想。層内だけなら 180s で解ける。

sequence-based ALNS + regret insertion(現行LNSの正しい強化)。

DRLは事前学習済みモデルを持ち込めるなら推論はCPUで可能だが、このコンペのカスタム環境に合わせた再学習が要するため5週間+GPUなしでは非現実的。

Details

(A) accessibility を第一級市民にする定式化 — 具体的処方

あなたの現行は「corridor_penalty(重み6.0)で今ある障害物を近視眼的に罰する」= accessibility を **目的関数の軟らかい項**として扱っている。これが天井の構造的な原因。文献が示す正しい3つの型:

構造的保証(推奨・最小リスク): wall-building/layer 構築で、コンテナを奥(y最大)から手前(y=-width/2)へ壁(層)単位で埋め、各層を閉じてから次の手前層を開く。robot position constraint(front-to-back)が構築規則として自動的に満たされ、搬入L経路は「まだ開いていない手前空間」を必ず通れる。Jiao et al. 2023 の robot position constraint の実装がこれ。あなたの「層規律(back_term 重み5.85)」は既にこの萌芽だが、**軟らかい選好**に留まっている。これを**ハードな層クローズ規則**に格上げする。FRONT_WALL が失敗したのは「密着」を強制したから;層構築は「密着」ではなく「前方空間の温存」を保証するので fill を削らない。

precedence graph(中リスク・TAP-Net流): 全荷物対で「AがBの搬入L経路を横切るなら $A < B$ 」の有向辺を張り(TAP-NetのLAB/RAB相当)、topo-sort可能な順序のみを候補にする。あなたのTOPO_REORDERの発展形。ただしTAP-Netの教訓通り、**precedence** を**順序制約としてだけ使うと弱い**(あなたの -4)。配置決定と統合する必要がある。

reachability を厳しくする(非推奨): Junqueira 2012 が「効率が落ちる」と実証済み。あなたの FRONT_WALL/TRANSPORT_MARGINがこれで、-11/-15 は理論通りの失敗。

(B) 探索の設計 — 評価関数を変えるのが本質

あなたのビーム/ロールアウト失敗(depth2 -32%)の根本原因は「体積で枝をランクした」こと。KDD 2026 MCTS論文と同じ轍。正しい枝評価 = そのプランを最後まで転がしたときの num_placed_items(足切り突破が最重要というあなたの結論と一致)、または「デッドロックまでの手数」。

LNSは骨格は正しいが removal/insertion が弱い。**sequence-based removal(連続ウィンドウ)+ regret-k insertion(先読み)**に変更(Voigt 2024 の実証ランキング準拠)。あなたのPACK_LNS_MID(中盤連続窓)は方向は正しかったが insertion が貪欲のままだった。

CDRの失敗(「短予算offlineシミュでstallが起きない→repair対象が現れない」)は、offlineシミュの予算配分の問題。**offlineシミュに実機ドリフトの分位点(悲観側)を織り込む**と、offline上でも大型荷物が stall するようになり、repair対象が顕在化する。

(C) sim-to-real — 分位点評価と選択的物理検証

一律悲観化(DRIFT_MODEL)は「臆病な順序を選び容積が半分空いたまま死ぬ」= 期待値最大化として間違い。正しくは:

分位点評価: 通常配置は期待値(現行の楽観的幾何)で評価、**クリティカル配置(item_idx=0 の大型)だけ悲観分位点(沈降ドリフトの上側)で搬入経路をチェック。**

選択的実物理: 8s/step 予算内で、大型荷物の直前に限り pybullet を短ステップ回して実ドリフト後の搬入経路を検証(Zhao 2022 の quasi-static equilibrium なら更に高速)。

残差モデル: offline幾何位置 → 実機沈降後位置 の残差を、ローカル評価スイートで収集して簡易回帰(線形/勾配ブースティング、CPUで軽い)し、クリティカル配置にだけ適用。

(D) ILP/CP の現実性

フル3D-BPPをCP-SAT/ILPで 180s は非推奨。 arXiv 2311.06314(CP for 3D packing)は「CPは大域最適を探すため探索空間が巨大でヒューリスティックに劣る」と結論。OR-Tools開発者も「CP-SATは幾何を理解せず noOverlapだけ、回転で変数爆発」とコメント([google/or-tools Discussion #5011](https://groups.google.com/g/or-tools-discussion/c/5011))。 [GitHub](#)

ただし部分問題には有効: 層構築で1層(2D配置)に落とせば、CP-SAT の noOverlap2D で層内最適化が 180s 内に回る。Nascimento, Queiroz & Junqueira (2020, Computers & Operations Research, "Practical constraints in the container loading problem") は ILP+CP のハイブリッドで実務制約付き SCLPの70%超を1時間で解く。時間予算的にフルは無理だが層単位なら現実的。

column generation は理論的に強いが、pricing問題(3Dパッキング)自体がNP困難で 180s には重い。非推奨。

(E) 最小リスクの次の一手 — 継続 vs 凍結の判断

現状評価:public 58.5 ~ もっともらしい天井の97%、十数系統が全滅、残り5週間、2コア・GPUなし。

判定:限定的継続を推奨。ただし単一の賭けに集中し、明確なストップロスを設定。

理由:

あなたが試した十数系統は**すべて「同じ貪欲構築ソルバの上での探索/修復」**であり、共通の失敗モード(貪欲プランナの1荷物局所最適が不動点)を共有している。これは「探索を変えても駄目」を意味するが「定式化を変えても駄目」は意味しない。未着手の本質的変更(wall-building構築規則への差し替え)が1つ残っている。文献(Junqueira 2012、Jiao 2023、Parreño 2008、Bonet Filella 2023)は、accessibility制約下では wall-building/front-to-back 構築が充填と両立する正攻法だと一貫して示す。あなたの現行は Extreme Point 貪欲で、この正攻法を実装していない。

DRL は本命だが5週間+GPUなしで棄却。ILP/CP フルも棄却。残る現実解が wall-building 一択。

Recommendations(優先順位付き・実装難易度/期待効果/リスク付き)

第1手(最優先・2週間):wall-building / maximal-space 構築規則への差し替え

内容:planner を Extreme Point 貪欲から **maximal-space + 層(壁)構築**(Parreño 2008)に差し替え、コンテナを奥→手前に層単位で埋め、層クローズ規則で front-to-back を**ハード保証**。目的は体積ではなく

num_placed_items(足切り突破)。

難易度:中〜高(planner中核の書き換え)。期待効果:大(天井の構造的原因を除去)。リスク:中(新規実装のバグ)。

判断基準(ストップロス): 2週間で、ローカル評価スイート26+2シーンのうち悪化シーン数が2以下かつ **net** で明確にプラスの「強い信号」が出るか。出れば本番投入、出なければ第1手を撤退。

第2手(第1手が効いたら・1週間):sequence-based ALNS + regret insertion で層順序を磨く

内容:層構築の層順序/層内荷物集合を、sequence-based removal + regret-k insertion で LNS(Voigt 2024 準拠)。位置は強制しない(局所最適を壊さない)。

難易度:中。期待効果:中(+1〜2点)。リスク:低。

第3手(並行・1週間):sim-to-real の分位点+選択的物理検証

内容:クリティカル配置(item_idx=0 大型)だけ悲観分位点で搬入経路をチェック、必要なら 8s 内で短pybullet 検証。offlineシミュにドリフト分位点を織り込み、CDRの repair対象を顕在化。

難易度:中。期待効果:中(sudden death を減らし完走シーンを増やす=配置率直結)。リスク:低。

第4手(層内最適化・オプション):CP-SAT で層内2D配置

内容:確定した1層内の2D noOverlap 配置を CP-SAT で局所最適化(180s のごく一部を割当)。

難易度:中。期待効果:小〜中。リスク:低。

やらないこと: フル3D ILP/CP、column generation、GPUなしでのDRLゼロから学習、これ以上の「制約を厳しくするフィルタ」(FRONT_WALL/TRANSPORT_MARGIN系)。すべて時間対効果が負。

凍結条件: 第1手が2週間で強い信号を出せなければ、58.5 の主枠 w3_planexec_ins_la_tb135.zip と2枠目 rest020.zip の提出物完全性(フォーマット・タイムアウト耐性・プロセス再起動時の状態再構築)を確定し、防御に回る。これが期待値最大の撤退。

Caveats

NEDO Challenge コンテスト2 自体の上位解法・スコア分布は未公開。 コンペ進行中(締切2026-10-19)のため、Qiita/Zenn/note/SIGNATE公式に当該コンペの解法記事は現時点で確認できなかった。「60+勢が何をしているか」は類似コンペ・文献からの**推定**であり、直接の裏取りではない。 [PR TIMES](#)

↑

Junqueira et al. 2012 の書誌には注意。 reachability制約(到達距離 δ)の結果を報告した論文について、二次情報間で帰属に食い違いがある。同著者(Junqueira, Morabito & Yamashita)は2012年に2本を出しており、(a) "MIP-based approaches for the container loading problem with multi-drop constraints", Annals of Operations Research 199(1):51-75, DOI 10.1007/s10479-011-0942-z、(b) "Three-dimensional container loading models with cargo stability and load bearing constraints", Computers & Operations Research 39(1):74-85, DOI 10.1016/j.cor.2010.07.017 がある。reachabilityは multi-drop 文脈で研究されるため (a) が自然だが、Bonet Filella et al. (2023) の参照では別帰属の可能性も示唆され、**どちらの論文が δ パラメータ実験を含むかは原論文PDFで確定すべき。**ただし「reachabilityを厳しくすると充填効率が落ちる」という定性的結論そのものは **Bonet Filella et al. (2023, EJOR) の verbatim 引用で確証済み。**

Jiao et al. 2023(CLP-RLS)の定量結果(充填率の具体的数値、robot制約による効率低下の%)は本文がペイウォールのため未確認。 制約の定義(pallet continuity / robot position)・解法名(wall-building tree search)・目的(loading length最小化)は要旨 verbatim で確証済みだが、正確な数値は原論文PDFの直接確認が必要。

あなたのローカル評価が本番に転移しにくいという自己診断は文献的にも妥当(弱い信号は符号反転しうる)。
第1手の判断は必ず「悪化シーン数」という強い信号で行うこと。

DRL系(PCT/TAP-Net/Attend2Pack/KDD 2026 MCTS)は理論的本命だが、いずれも**GPU学習前提**。5週間・GPUなしでは推論転用すら環境差で困難なため、本報告では意図的に優先度を下げている。もし将来GPUと数ヶ月があれば、PCT + precedence graph(TAP-Net流)+ テスト時 deliberate planning が最有力の再挑戦経路。